



UE22CS352B - Object Oriented Analysis & Design

**TRAFFIC VIOLATION AND ACCIDENT RECORD
MANAGEMENT**

Submitted by:

NAME: GOWTHAMI LJ SRN: PES2UG22CS204

NAME: HARI SHANKAR SRN: PRES2UG22CS213

NAME: HARSHITHA L SRN: PES2UG22CS219

NAME: HARSHITHA MP SRN: PES2UG22CS220

Faculty Name: DR. SUJA C M

January - May 2025

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY**

(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

Problem Statement:

In modern urban environments, the rising volume of vehicular traffic has made it increasingly difficult for authorities to manage and monitor road safety. Traditional manual processes for recording traffic violations and accident reports are inefficient, error-prone, and lack real-time accessibility.

This project addresses the need for a **centralized, digital platform** that facilitates the recording, management, and retrieval of traffic-related data. The proposed system provides:

- A **web-based login system** for both **drivers** and **traffic officers**.
- **Drivers** can view their registered vehicle details, recorded traffic violations, and accident history.
- **Officers** can securely log new traffic violations and accident incidents.
- All interactions are processed through a **Java backend** (without servlet/Tomcat) using a lightweight web framework, and follow the **ModelView-Controller (MVC)** design pattern.
- The system uses a **MySQL database** to store and manage all relevant records securely.

By digitizing the workflow and providing dedicated dashboards for different users, this system enhances transparency, efficiency, and accessibility in traffic management operations, ultimately contributing to improved road safety and better data-driven decision-making.

Key Features:

1. Role-Based Login System

- Separate login options for **Drivers** and **Officers**
- Secure login with username, password, and user role selection
- Role-based redirection to dedicated dashboards

2. Driver Dashboard

- View personal **vehicle information**
- View list of **traffic violations** recorded against the vehicle
- View **accident history** (if any) involving the driver's vehicle

3. Officer Dashboard

- Add traffic violations by entering vehicle number, violation type, and fine □
Report accidents with details such as location, severity, and involved driver/vehicle □
□ View previously recorded entries .

4. MySQL Database Integration

- Persistent storage of users, vehicles, violations, and accidents
- Linked relational tables for **drivers, vehicles, officers, violations**, and accidents. □
Ensures **data consistency** and **fast querying**.

5. Web-Based Frontend Interface

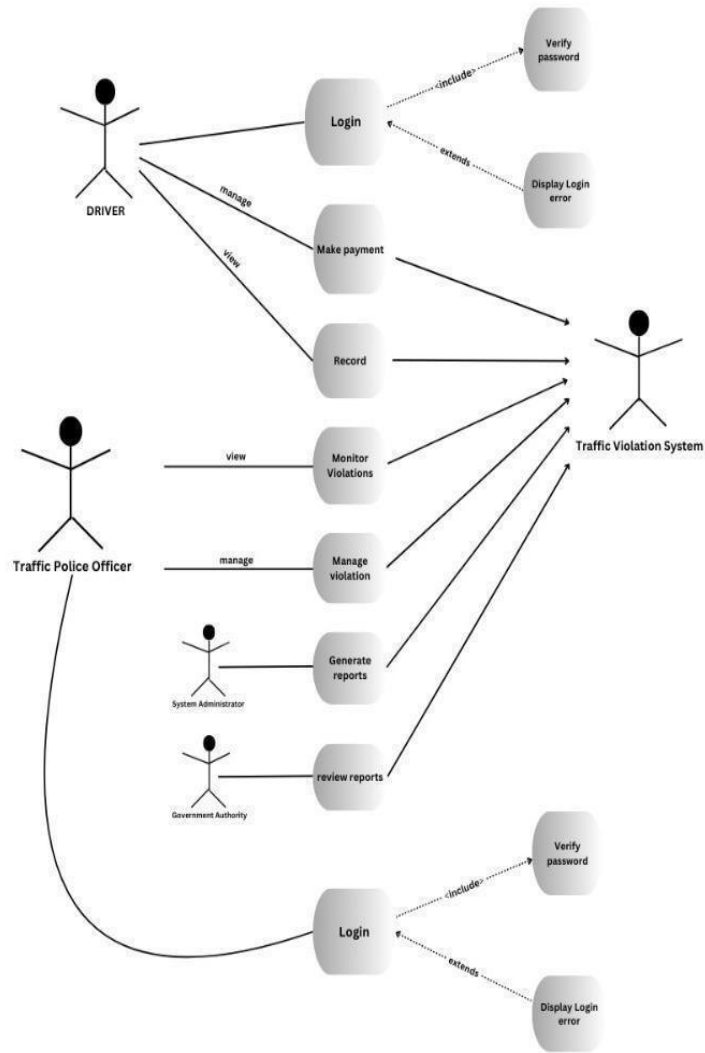
- Responsive and attractive **user interface**
- Form-based input system for login, signup, and data entry
- Intuitive navigation across role-based pages

6. Java Backend (MVC Architecture)

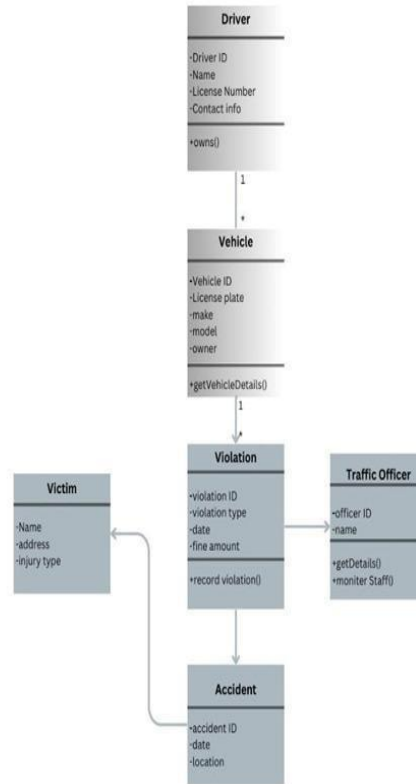
- **Model-View-Controller (MVC)** pattern separates business logic from presentation
- Built using **Spark Java** for lightweight server functionality (no Servlet or Tomcat) □
Backend routes handle **login, signup, form submissions**, and redirection

Models:

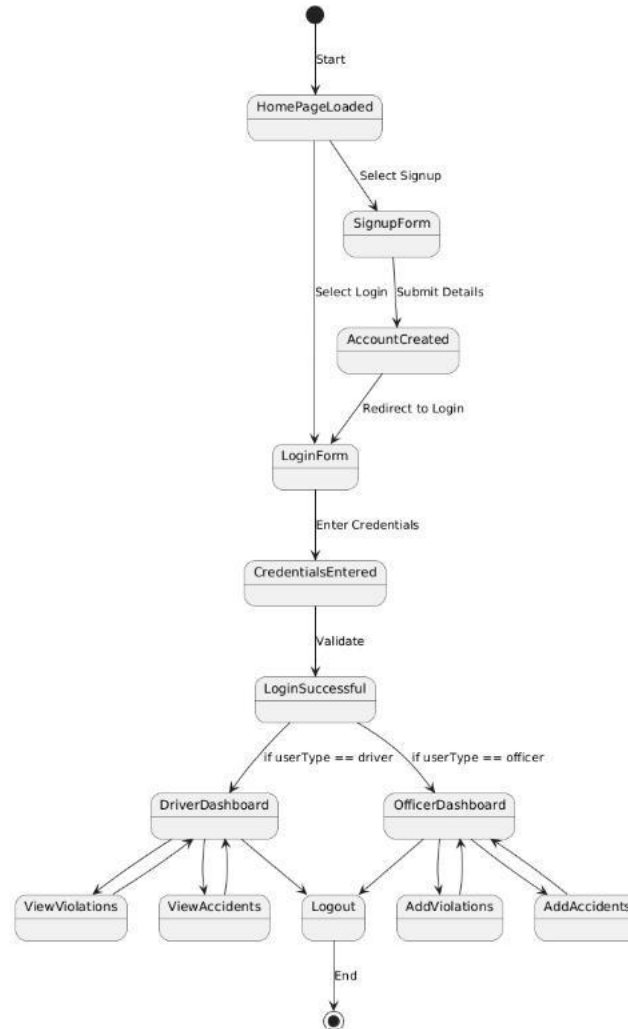
Use Case Diagram:



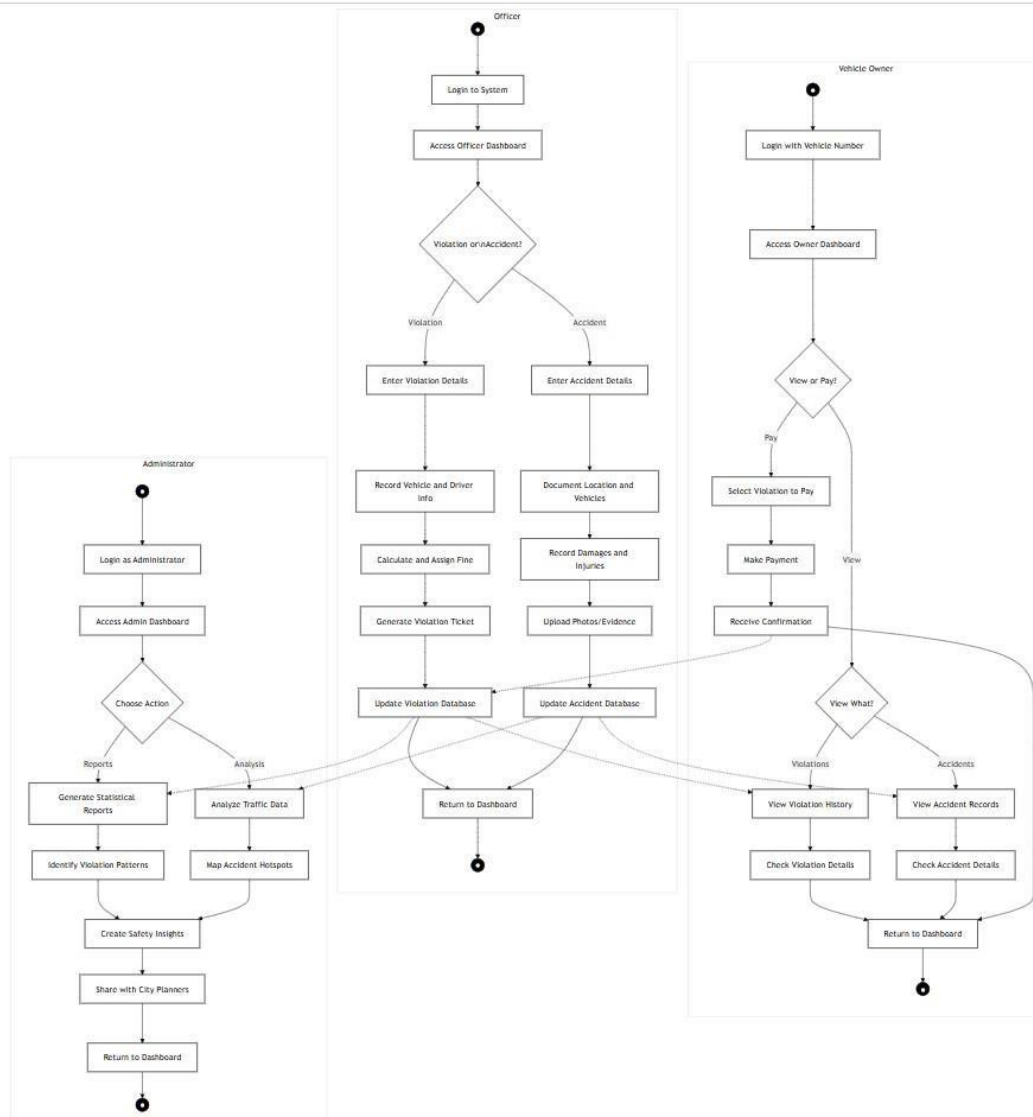
Class Diagram:



State Diagram:



Activity Diagrams:



Architecture Patterns

Model-View-Controller (MVC) Architecture

MVC is a software architectural pattern that separates an application into **three interconnected components**:

- **Model**: Manages data and business logic
- **View**: Handles the presentation/UI
- **Controller**: Manages user input and communication between model and view

This pattern helps in organizing code effectively, increasing **reusability, scalability, and maintainability**.

In Our Project

Model layer: Contains logic for accessing and interacting with the database.

Files:

- `UserDAO.java`: Contains logic for registering and authenticating users.
- `Database.java`: Handles connection to the MySQL database. □ `Driver.java`, `Officer.java`: Represent data entities.

Example:

- When a new user signs up, `UserDAO.register ()` inserts their details into the database.

View layer

The **View** in this project is implemented using modern **HTML, CSS, and JavaScript**, replacing the earlier terminal-based Java UI. It is responsible for presenting the

interface to the user, collecting user inputs via forms, and displaying feedback such as login options, registration status, and dashboard navigation. Each user role — driver or officer — is provided with a dedicated web interface for a seamless and role-specific experience. The View layer communicates with the backend controllers through HTTP POST requests, making the application more interactive, user-friendly, and visually appealing. This approach also promotes the separation of concerns, where the frontend UI logic is completely decoupled from the business and database logic.

Controller Layer

Acts as an intermediary between View and Model. Processes user input, calls the model, and returns the response.

□ Files:

- `Main.java`: Entry point that starts the server and sets up route handling.
- `LoginController.java`: Handles POST requests to `/login` and redirects to officer/driver dashboard.
- `SignupController.java`: Handles POST requests to `/signup` and writes user data to DB via `User DAO`. □ **Example:**
- When a user submits the login form, `Login Controller` validates credentials using the `User DAO (Model)` and then redirects to the correct View (`officer.html` or `driver.html`).

DESIGN PRINCIPLES

This project applies several key object-oriented and architectural design principles to ensure clean, modular, and maintainable code:

1. Separation of Concerns

Each part of the system (Model, View, and Controller) is responsible for a specific functionality:

- The **Model** handles database operations.
- The **View** is responsible for user interaction and UI.
- The **Controller** acts as the intermediary between the user interface and backend logic.

This makes the system easier to debug, extend, and test.

2. Single Responsibility Principle (SRP)

Each class or component in the project has only one responsibility:

- `UserDAO.java` handles only database interaction for users.
- `LoginController.java` is responsible only for login routing logic.

3. Open/Closed Principle

The code is **open for extension but closed for modification**:

- New features like report generation or additional roles can be added with new files/modules without modifying existing ones.

4. Encapsulation

- Database access is abstracted using `User DAO`, preventing direct access to SQL logic from other parts of the code.

5. Modularity

- Logic is modularly split between layers and packages (e.g., `controller`, `model`), which enhances maintainability and team collaboration.

Design Patterns

The project adopts several standard design patterns to ensure better structure, flexibility, and maintainability of the codebase.

1. Model-View-Controller (MVC) Pattern (Architecture Pattern)

- **Model:** Java classes that handle data and database interactions (`UserDAO`, `Database`)
- **View:** frontend pages
- **Controller:** Java route handlers using Spark Java (`Login Controller`, `Signup Controller`)
 - > Promotes separation of concerns and better modularization.

2. DAO (Data Access Object) Pattern

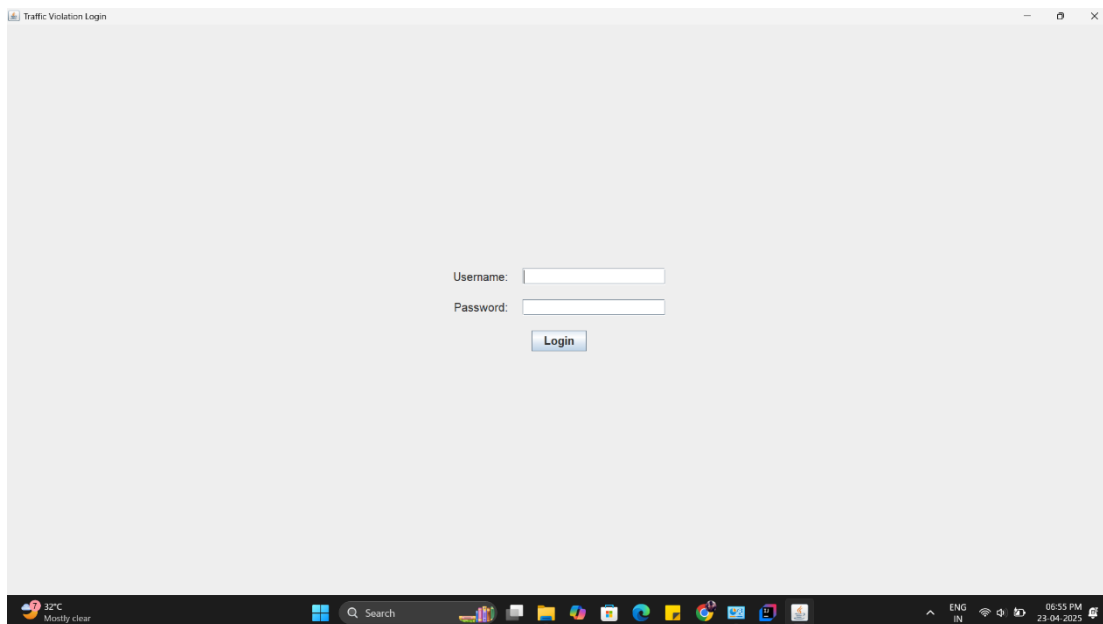
- Implemented in: `UserDAO.java`
- Purpose: Abstracts and encapsulates all access to the data source, making it easy to manage database operations like login and signup. > Promotes reusable, cleaner, and testable data access code.

3. Front Controller Pattern (Simplified via Spark Java)

- `Main.java` acts as a central controller that sets up all the application routes (`/login`, `/signup`, etc.)
- This follows the **Front Controller** concept where a single handler delegates to different controllers/routes.
>provides a unified entry point for all web requests.

Github link to the Codebase:

Screenshots of UI:



Officer Dashboard - Traffic Management System

Driver ID:

First Name:

Last Name:

Date of Birth (YYYY-MM-DD):

Gender:

Address:

City:

State:

House No:

Street:

Add

Driver ID	First Name	Last Name	DOB	Gender	Address	City	State	House No	Street
1	Gowthami	Jeevanna	2004-07-05	Female	Lingappanakoppalu	Halebidu	KT	91	clock st
2	Shekar	raj	1908-09-09	Male	Jay nagar	Benagaluru	KT	234	church st
34	Robin	Hood	1989-09-19	Male	JP nagar	Bengaluru	KT	23	church st
46	AA	BB	1990-08-08	Male	typ	tyt	bdhyc	89	tyt
50	shri	Raj	1907-08-09	Male	krnkvn	ejfb	ejfb	45	otbn
56	Harsha	Shetty	1999-03-04	Male	JP nagar	Bengaluru	KT	67	Mg st
66	ggu	ngb	1999-09-08	Male	ulur	kurn	lbn	133	ndtr
78	shoban	babu	1999-05-04	Male	12th cross, church st	wzsq	Andhra Pradesh	56	church st
100	Karthik	Aryan	1890-03-04	Male	Electronic city	Bengaluru south	KT	90	MG road
200	Allu	Aryan	1990-09-08	Male	jay nagar	bengaluru	KT	38	Mg st

Next

Officer Dashboard - Traffic Management System

Vehicle No:

Model:

Type:

Year:

DriverID

Add

Vehicle No	Model	Type	Year	DriverID
KA34T0689	kia	4 wheeler	2020	34
56	BMW	4 wheeler	2020	100
AP34T0245	Benz	4 wheeler	2024	78
KA09QA4567	kia	4 wheeler	2009	200
KA09QW2345	sougr	pkb	2013	50
KA09AF0984	Benz	4 wheeler	2022	1
KA04AF2004	Benz	4 wheeler	2023	2

Next

Officer Dashboard - Traffic Management System

Victim ID:

First Name:

Last Name:

Date of Birth (YYYY-MM-DD):

Gender:

Address:

AccidentID:

Add

Victim ID	First Name	Last Name	DOB	Gender	Address	OfficerID
1	Harshiha	Changappa	2004-06-06	Female	Electronic city	1
26	Raj	Raj	1999-07-07	Female	hcosur	25
67	Munayappa	raju	1990-08-16	Male	linganakopalu	56
70	sangeeth	reddy	1900-09-09	Male	vizag	50
90	venkat	raju	1990-03-03	Male	kupam	69
96	kariyappa	sam	1999-05-09	Male	wedborgf	100

Submit

32°C Mostly clear

Search

ENG IN

08:56 PM 23-04-2025

Driver Dashboard

Search Violations

Driver ID:

Vehicle No:

First Name:

Search

No violations found

32°C Mostly clear

Search

ENG IN

08:56 PM 23-04-2025

Individual contributions of the team members:

Name	Module worked on
GOWTHAMI L J	Controller & Routing Logic
HARI SHANKAR	Integration, Testing, and Reporting
HARSHITHA L	Database Integration & Data Access Layer
HARSHITHA M P	User Interface